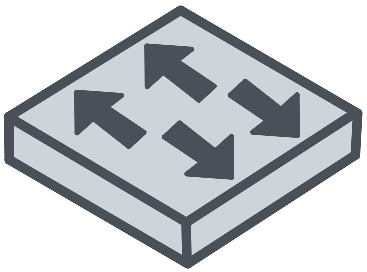
FlashAttention - IO-Aware Attention

Softmax(QK^T)V

Presented by: Adin Mauer
Date: January 21, 2025

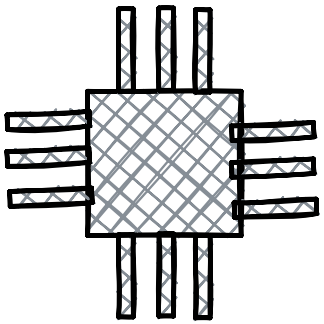
Keeping the GPU busy

HBM



1.5 TB/s

SRAM/SM



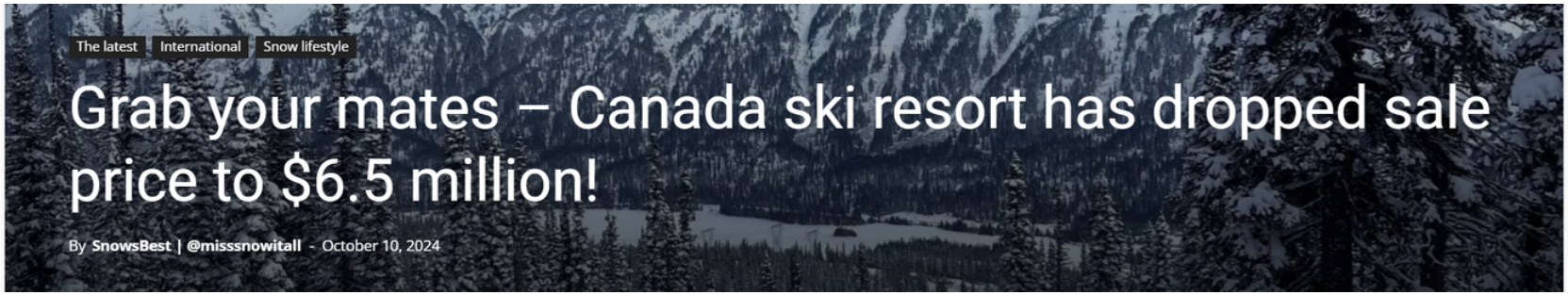
19.5 FP64 (8 bytes) TFLOPS

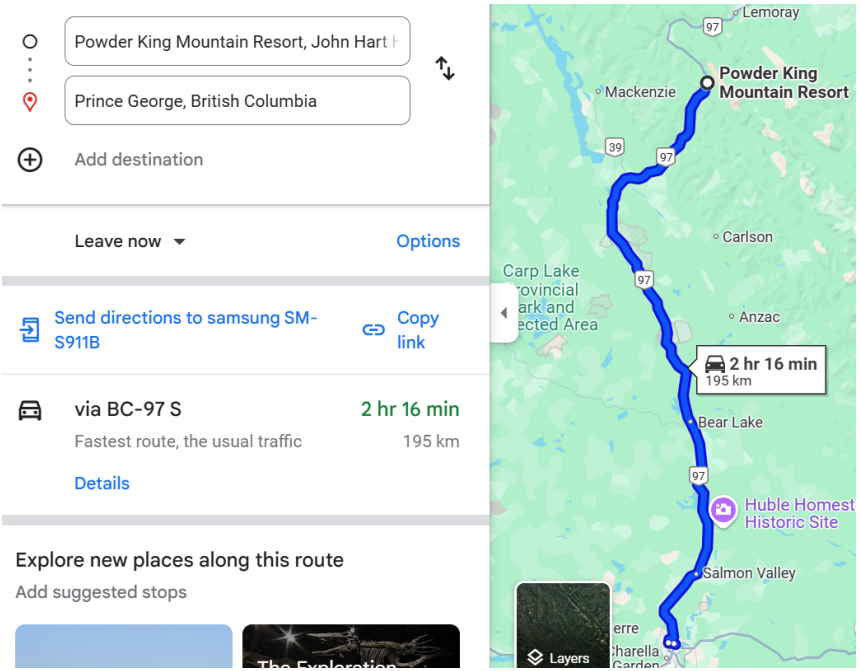
Arithmetic Intensity

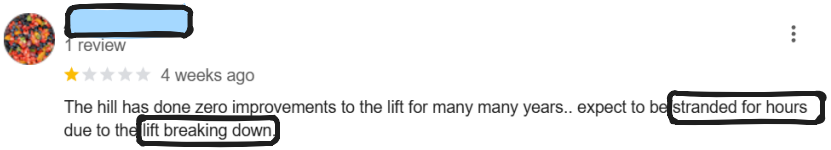
19.5/(1.5/8)=104

Conclusion: Need 104 operations on each FP64 element (at least) to keep processor continuously busy

* from Stephen Jones, "How GPU Computing Works", GTC 2021

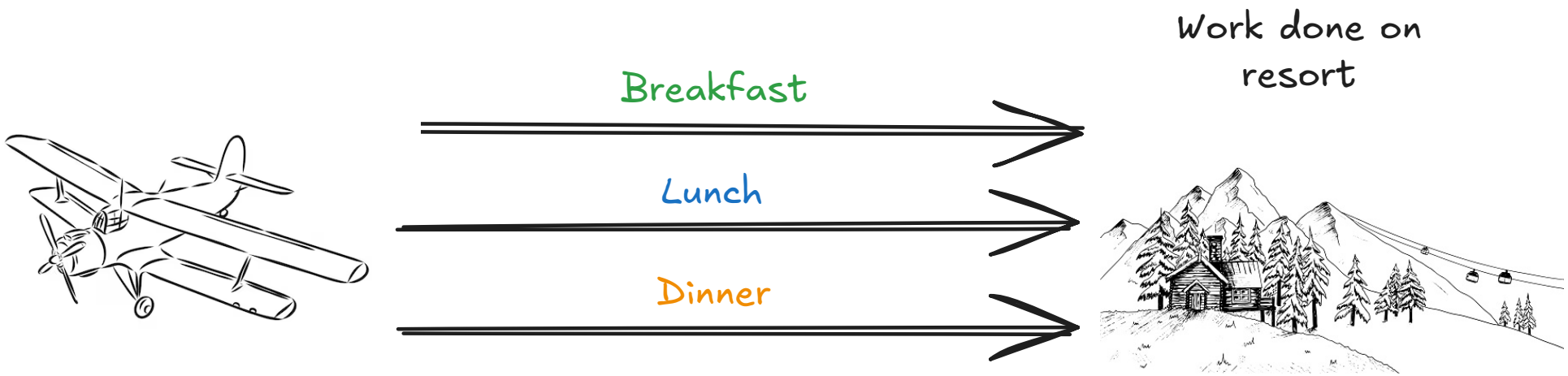

The latest
International
Snow lifestyle
Grab your mates – Canada ski resort has dropped sale price to $6.5 million!
By SnowsBest | @misssnowitall - October 10, 2024


Powder King Mountain Resort, John Hart
Prince George, British Columbia
Add destination
Leave now
Options
Send directions to samsung SM-S911B
Copy link
via BC-97 S
2 hr 16 min
Fastest route, the usual traffic
195 km
Details
Explore new places along this route
Add suggested stops
Lemoray
Powder King Mountain Resort
Mackenzie
Carlson
Carp Lake
Anzac
2 hr 16 min
195 km
Bear Lake
Huble Homest Historic Site
Salmon Valley
Layers


1 review
4 weeks ago
The hill has done zero improvements to the lift for many many years.. expect to be stranded for hours due to the lift breaking down.

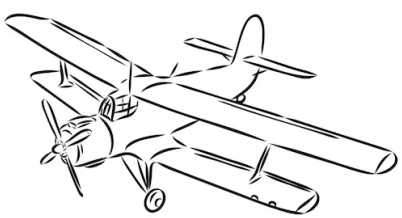
100 employees
3 meals a day

3 daily trips - inefficient



Very costly transportation - solution:
Kernel Fusion

100 employees
3 meals a day



1 daily trip

3 meals

Work done on resort

Dinner would spoil by the evening - solution...

100 employees
3 meals a day

1 weekly trip

Ingredients for 1 week

Work done on resort

Hire chef, build kitchen, ship in ingredients once a week

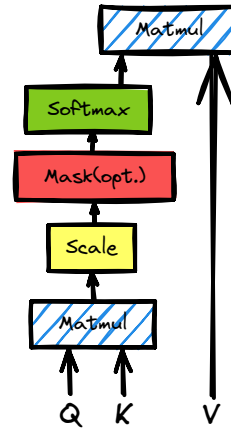
Too simplistic? Maybe..

But it all comes down to having a correct mental model of the system paired together with smart operation management

FlashAttention - IO-Aware Attention

The Problem

- HBM accesses in Transformers scale quadratically with increasing sequence length
- This, combined with the fact that the algorithm is memory-bound (not computer-bound), creates a problem



$$\text{Attention}(Q,K,V)=\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Compute-Bound

Means that the algorithm runtime is constrained by the actual computation. Improvement in computation efficiency will result in improvement in the algorithm performance

Memory-Bound

Means that even if computation efficiency increases, the algorithm will not perform better, because it is being limited by IO and memory considerations. A possible solution is an increase in arithmetic intensity

Keeping the GPU busy

HBM

1.5 TB/s

SRAM/SM

19.5 FP64 (8 bytes) TFLOPS

Arithmetic Intensity

19.5/(1.5/8)=104

Conclusion: Need 104 operations on each FP64 element (at least) to keep processor continuously busy

* from Stephen Jones, "How GPU Computing Works", GTC 2021

Previous Solutions

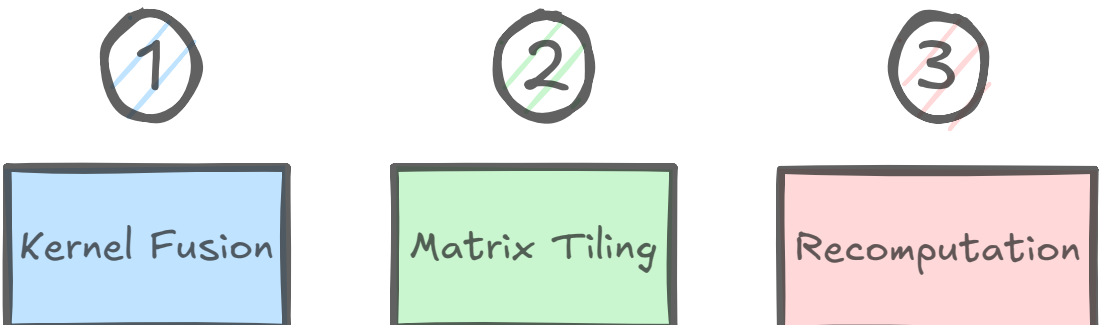
Low-Rank Solutions

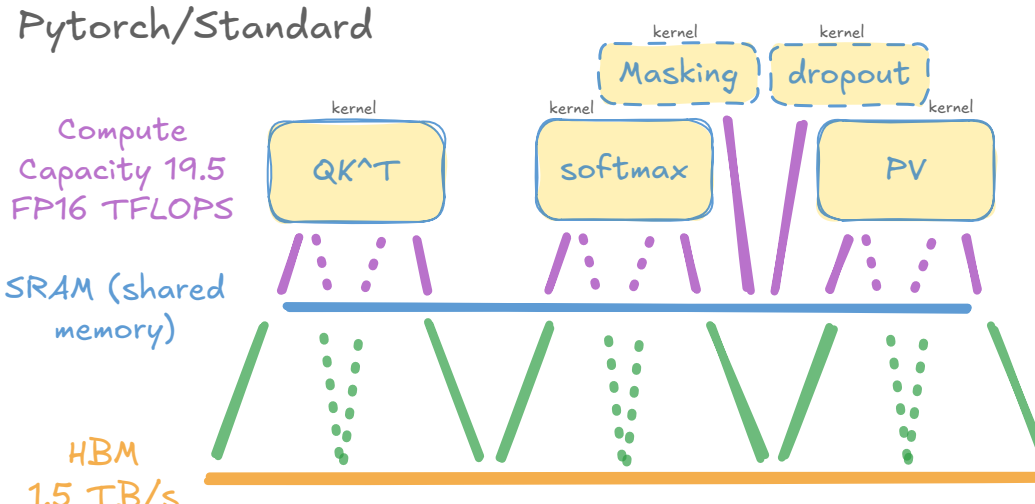
Sparsity (keyformer)

Other optimizations

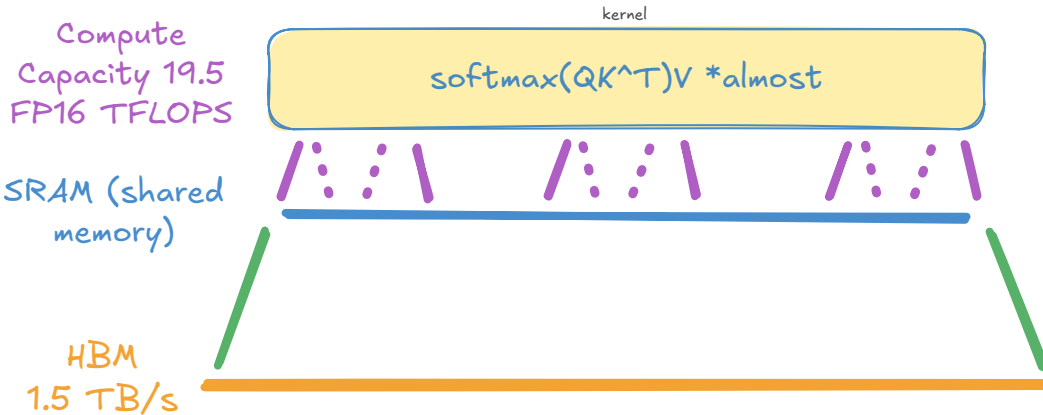
They all improve FLOPS, but not memory (HBM) access scaling

FlashAttention Solutions

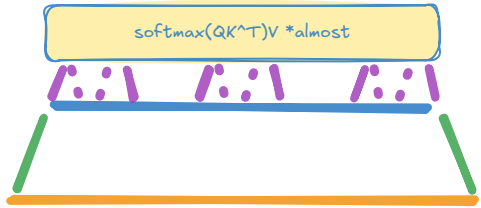



Pytorch/Standard
Compute Capacity 19.5 FP16 TFLOPS
SRAM (shared memory)
HBM 1.5 TB/s
kernel
QK^T
kernel
softmax
kernel
Masking
kernel
dropout
kernel
PV

FlashAttention
KERNEL FUSION

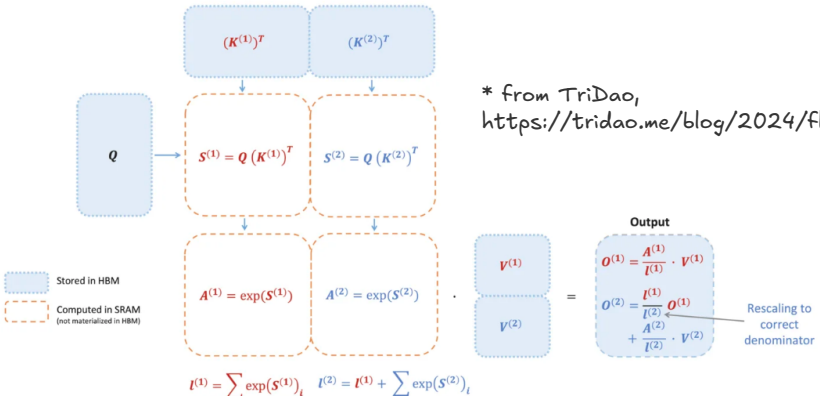


How can we achieve this?
TILING (almost)



Tiled Softmax

$$m(x) := \max_i \; x_i, \quad f(x) := \left[e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}\right], \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$



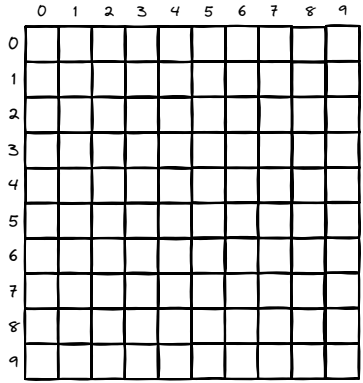
* from TriDao, https://tridao.me/blog/2024/flash3/

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)} \; x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

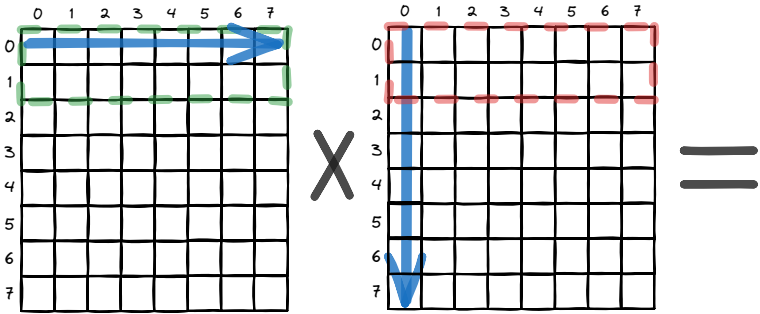
$$m(x) = m(\left[x^{(1)} \; x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)}) - m(x)} f(x^{(1)}) \quad e^{m(x^{(2)}) - m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)} \; x^{(2)}\right]) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}), \quad \mathrm{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

Lets quickly talk about 'tiling'



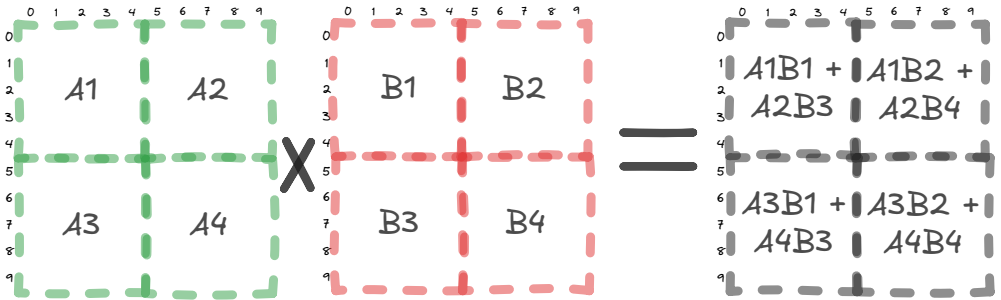
Lets say shared memory (or LLC) is of size 32



Every element calculation causes several cache misses.

Also, data is rejected from the cache without ever being used? Transported to/from memory for nothing.

Lets say shared memory is of size 32



Now we are maximizing the arithmetic intensity of each time data is in memory, maximizing the lowest level cache size.

Tiling - in general - is implemented by following 2 principle rules:

1. Once a memory level is completely filled with data, execute the maximum amount of work that is possible to accomplish with that data (without crossing real dependencies with other data elements)

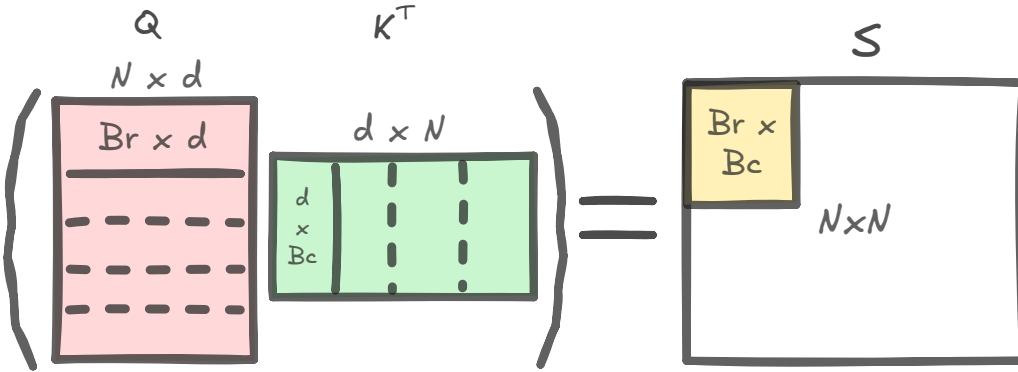
2. Fill memory levels in a order and data access pattern that minimizes that total number of data distributions and communications required to complete the computation

Tiling - in general - is implemented by following 2 principle rules:

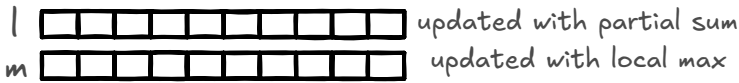
By following those 2 principles, it is possible to extend tiling implementations to nested tiling implementations.

** Maybe hint toward multi-gpu implementation?

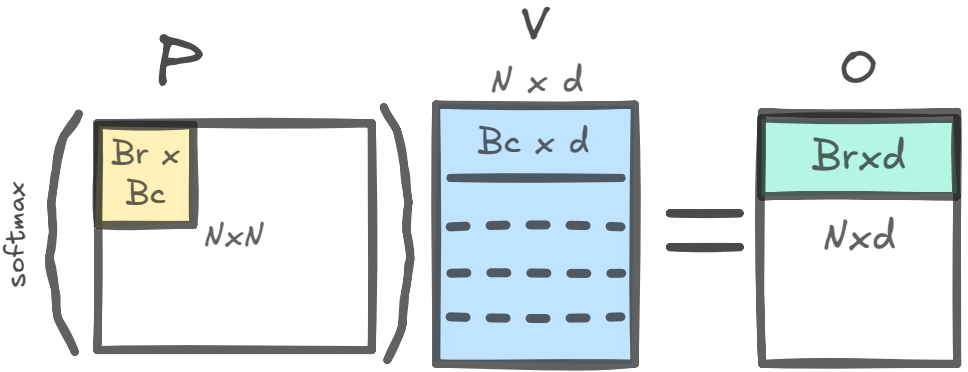
Flash Attention Tiling
segments K, Q and V matrices
in the sequence (N) dimension



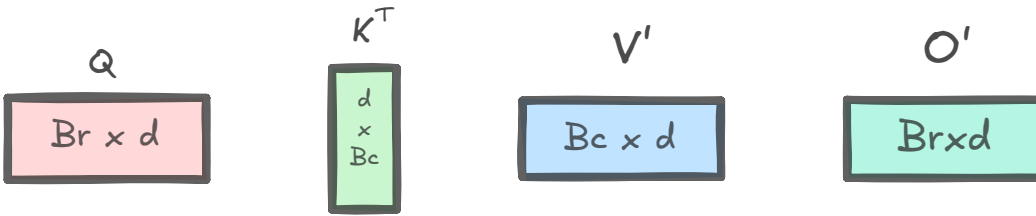
Flash Attention Tiling segments K, Q and V matrices in the sequence (N) dimension



* Will revisit later



Flash Attention Tiling segments K, Q and V matrices in the sequence (N) dimension



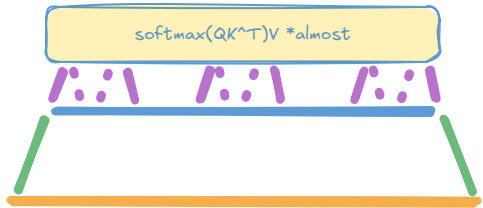
For Br,Bc=M/(4d), total memory footprint is 4*(M/(4d))*d=M
Where M is the SRAM size, so every kernel completely fits on the SRAM shared memory

Back to this. Why (almost) ?

Because of re-computation
(and numerical precision)

Back to this. Why (almost) ?

Because of re-computation
(and numerical precision)



Further Extensions

Block Sparsity

Same as FlashAttention, but only mask block with $B \in \{0,1\}^{N/B_c \times N/R_c}$

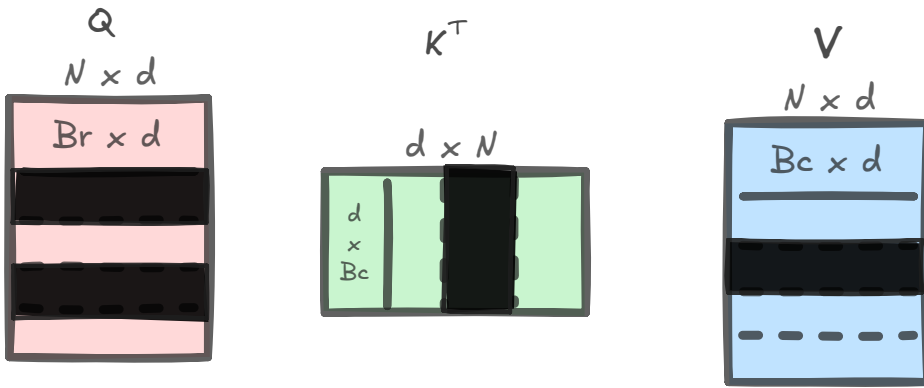
So you only compute non-zero blocks.

Has been shown to improve runtime proportional to fraction of non-zero blocks in mask

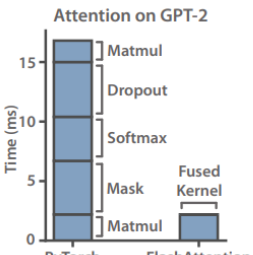
Has shown performance on par with FlashAttention

Further Extensions

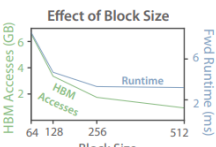
Block Sparsity

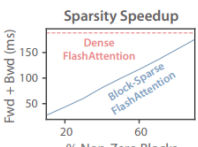


Results



Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3





Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

Results

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
Block-sparse FlashAttention	56.0	**63.1**

Table 4: GPT-2 small with FlashAttention, with 4× larger context length compared to Megatron-LM, is still 30% faster while achieving 0.7 better perplexity. Training time on 8×A100 GPUs is reported.

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

Challenges and Future work

1. Engineering work - this algorithm does not generalize and must be written as custom-tailored CUDA kernels per-architecture/system.

2. Multi-GPU IO aware methods

Good to know:

- FlashAttention 2.0 was released in July 2023 and uses optimized CUDA kernels and smarter thread/block scheduling to increase performance by 1.3x-2x

- FlashAttention 3.0 (beta) was released in July 2024 to utilize low-precision (FP8), advanced CUDA features, and tailor the implementation specifically for the Hopper H100 GPU to increase performance by 1.2x-1.3x relative to V2.